



sim, ficou claro "o que" fazer, mas não ficou claro "como" fazer. Digamos assim, o que deve ser feito para poder ter essa edição de maneira fluida tal qual é no canva

Ótima pergunta! Vou detalhar o "como" técnico, passo a passo, focando na mecânica que faz a fluidez acontecer.

O Segredo da Fluidez: Arquitetura Reativa Unidirecional

A fluidez do Canva vem de um princípio simples: todo o estado vive em um único lugar, e tanto o canvas quanto o painel são apenas "espelhos" desse estado. Quando você muda em um lugar, todos os outros atualizam automaticamente.

Vou te mostrar a mecânica exata.

1. Estrutura do Estado Global

Primeiro, você precisa de um único objeto de estado que representa tudo.

Como estruturar:

```
ESTADO GLOBAL
├─ document
│   ├─ id: "post-123"
│   ├─ width: 1080
│   ├─ height: 1080
│   ├─ background: { type: "color", value: "#FFFFFF" }
│   └─ textBoxes: [
│       {
│         id: "box-1",
│         x: 100,
│         y: 150,
│         width: 400,
│         height: 60,
│         rotation: 0,
│         runs: [
│           {
│             id: "run-1",
│             text: "Black ",
│             formatting: {
│               fontFamily: "Inter",
```

```
|
|   |       fontSize: 48,
|   |       fill: "#000000",
|   |       fontWeight: 400,
|   |       shadow: null
|   |     }
|   | },
|   | {
|   |   id: "run-2",
|   |   text: "Friday",
|   |   formatting: {
|   |     fontFamily: "Inter",
|   |     fontSize: 48,
|   |     fill: "#FF0000",
|   |     fontWeight: 700,
|   |     shadow: { color: "#000", blur: 4, offset: { x: 2, y: 2 } }
|   |   }
|   | }
|   ]
| }
| ]
└─ images: [...]

└─ selection
    └─ type: "text-box"
        └─ id: "box-1"
```

Como implementar tecnicamente:

Use uma biblioteca de gerenciamento de estado reativo:

- **Zustand** (React) — mais simples, menos boilerplate;
- **Jotai** (React) — estado atômico, muito performático;
- **Redux Toolkit** (React) — mais robusto, mais código;
- **Signals** (Solid, Preact, Vue) — reatividade nativa.

Exemplo com Zustand (conceitual):

1. Crie uma "store" que contém:
 - O estado (document + selection)
 - Funções para atualizar o estado (actions)
2. Exporte hooks para os componentes usarem:
 - useDocumentStore() → acessa o estado
 - useDocumentStore.getState() → acessa de fora do React

2. Mecanismo de Re-renderização

Aqui está o coração da fluidez: fazer canvas e painel atualizarem automaticamente quando o estado muda.

Como funciona:

```
ESTADO MUDA
↓
Biblioteca de estado notifica todos os "ouvintes"
↓
Componentes React que usam aquele estado são marcados para re-renderizar
↓
React re-renderiza apenas os componentes afetados
↓
Canvas (Konva) recebe novas props → re-renderiza
↓
Painel recebe novas props → atualiza inputs
```

Como implementar:

No Canvas (React-Konva):

```
1. Cada componente (TextBox, Image, etc.) "assina" o estado:
const { document } = useDocumentStore()

2. Quando o estado muda, React chama o componente novamente:
function TextBoxComponent({ boxId }) {
  const { document } = useDocumentStore()
  const box = document.textBoxes.find(b => b.id === boxId)

  // Se box mudou, React-Konva atualiza o node do Konva automaticamente
  return <Text text={box.runs[0].text} fill={box.runs[0].formatting.fill} />
}
```

No Painel:

```
1. Painel "assina" a seleção:
const { selection, document } = useDocumentStore()

2. Se selection.id muda, painel carrega novo elemento:
useEffect(() => {
  if (selection.id) {
    const box = document.textBoxes.find(b => b.id === selection.id)
    setLocalRuns(box.runs) // Carrega trechos no painel
  }
}, [selection.id])

3. Inputs do painel são controlados pelo estado:
<input
  value={localRuns[0].text}
  onChange={(e) => {
```

```
// Atualiza estado global
updateTextBoxRun(selection.id, 0, { text: e.target.value })
}}
/>
```

3. Mecânica de Edição em Tempo Real

Aqui está o truque específico que faz a edição ser fluida:

Problema que você teve antes:

- Overlay DOM + canvas = dois estados separados;
- Sincronizar = complexo, bugs de timing;
- Resultado = texto no DOM $\neq$ texto no canvas.

Solução:

- Não existe estado local no input do painel;
- O input é controlado diretamente pelo estado global;
- Cada `onChange` dispara uma ação que atualiza o estado;
- Canvas e painel ambos leem do mesmo estado.

Fluxo exato (passo a passo):

PASSO 1: Usuário clica na caixa de texto no canvas

↓

Evento `onClick` da `TextBoxComponent` dispara:

`selectElement({ type: "text-box", id: "box-1" })`

↓

Estado `global.selection = { type: "text-box", id: "box-1" }`

↓

React notifica Painel que `selection` mudou

↓

Painel lê `document.textBoxes.find(id: "box-1")`

↓

Painel preenche inputs com os valores atuais

PASSO 2: Usuário digita "P" no input do painel

↓

Evento `onChange` do input dispara:

`updateTextBoxRun("box-1", 0, { text: "P" })`

↓

Estado `global.document.textBoxes[^0].runs[^0].text = "P"`

↓

React notifica:

- `TextBoxComponent` (canvas) → re-renderiza com novo texto
- Input do painel → atualiza `value` para "P"

↓

Usuário vê "P" no canvas INSTANTANEAMENTE

PASSO 3: Usuário continua digitando "romoção"

↓

Mesmo fluxo, caractere por caractere

↓

Canvas atualiza a cada letra

↓

Sensação de "edição direta no canvas"

Implementação técnica (exemplo conceitual):

```
// No Pannel
function TextPanel() {
  const { selection, document, updateTextBoxRun } = useDocumentStore()

  // Pega a caixa selecionada
  const box = document.textBoxes.find(b => b.id === selection.id)
  if (!box) return <div>Selecione um elemento</div>

  return (
    <div>
      {box.runs.map((run, index) => (
        <div key={run.id}>
          {/* Input de texto controlado pelo estado */}
          <input
            type="text"
            value={run.text} // ← Valor vem do estado global
            onChange={(e) => {
              // ← Atualiza estado global diretamente
              updateTextBoxRun(box.id, index, { text: e.target.value })
            }}
          />

          {/* Botão de cor */}
          <button
            onClick={() => {
              const newColor = prompt("Cor (hex):")
              updateTextBoxRun(box.id, index, {
                formatting: { fill: newColor }
              })
            }}
            style={{ backgroundColor: run.formatting.fill }}
          >
            ☐
          </button>
        </div>
      ))}
    </div>
  )
}
```

O segredo: O value do input **sempre** vem do estado global. Não há estado local. Isso garante que canvas e painel estão **sempre sincronizados**.

4. Performance: Evitando Re-renderizações Desnecessárias

Para ser fluido, não pode travar. Aqui estão os truques:

a) Memoização de componentes

Use `React.memo()` em componentes que renderizam muitos elementos:

```
const TextBoxComponent = React.memo(({ boxId }) => {
  const { document } = useDocumentStore()
  const box = document.textBoxes.find(b => b.id === boxId)

  return <Group>...</Group>
})
```

Isso faz o React pular re-renderização se as props não mudaram.

b) Seletores granulares no estado

Em vez de:

```
const { document } = useDocumentStore() // ← Assina tudo
```

Use:

```
const box = useDocumentStore(state =>
  state.document.textBoxes.find(b => b.id === boxId)
) // ← Assina apenas esta caixa
```

Assim, quando outra caixa muda, este componente **não re-renderiza**.

c) Batch de atualizações

Se o usuário faz várias mudanças de uma vez (ex: aplicar negrito + cor):

```
updateTextBoxRun(boxId, index, {
  formatting: {
    fontWeight: 700,
    fill: "#FF0000"
  }
})
```

// ← Uma única atualização, não duas

d) Debounce para operações caras

Se você tem algo pesado (ex: calcular sombra complexa):

```
const debouncedUpdate = useMemo(
  () => debounce((updates) => {
    updateTextBoxRun(boxId, index, updates)
  }, 100),
```

```
[boxId, index]
)

// Use debouncedUpdate em vez de updateTextBoxRun direto
```

5. Mecânica de Seleção e Bounding Box

Para a seleção ser fluida:

a) Estado de seleção é único

```
ESTADO.selection = { type: "text-box", id: "box-1" }
```

b) Canvas reage à seleção

```
function SelectionBox() {
  const { selection, document } = useDocumentStore()

  if (!selection.id) return null

  const element = selection.type === "text-box"
    ? document.textBoxes.find(b => b.id === selection.id)
    : document.images.find(i => i.id === selection.id)

  return (
    <Rect
      x={element.x - 2}
      y={element.y - 2}
      width={element.width + 4}
      height={element.height + 4}
      stroke="#00A8FF"
      dash={[5, 5]}
    />
  )
}
```

c) Clique no fundo limpa seleção

```
function Canvas() {
  const { clearSelection } = useDocumentStore()

  return (
    <Stage
      onClick={(e) => {
        // Se clicou no stage (não em um elemento)
        if (e.target === e.target.getStage()) {
          clearSelection()
        }
      }}
    >
```

```

    ...
  </Stage>
)
}

```

6. Mecânica de Drag-and-Drop Fluida

Para o arrastar ser suave:

a) Use draggable nativo do Konva

```

<Group
  draggable
  onDragEnd={(e) => {
    updateTextBox(boxId, {
      x: e.target.x(),
      y: e.target.y()
    })
  }}
>

```

b) Atualize estado apenas no final (onDragEnd)

```

Durante o drag, Konva move visualmente (sem atualizar estado)
↓
No onDragEnd, você atualiza o estado
↓
Estado muda → canvas re-renderiza na nova posição
↓
Próximo drag começa da posição correta

```

Por que não atualizar a cada pixel?

- Seriam centenas de atualizações de estado por segundo;
- React ia re-renderizar tudo, travando;
- Konva já faz drag visualmente, não precisa de estado.

c) Sincronize painel durante o drag (opcional)

Se quiser que o painel mostre X, Y atualizando enquanto arrasta:

```

<Group
  draggable
  onDragMove={(e) => {
    // Atualiza estado (painel vai mostrar em tempo real)
    updateTextBox(boxId, {
      x: e.target.x(),
      y: e.target.y()
    })
  }}
>

```



```

    })
  }}
  onDragEnd={ (e) => {
    // Garante estado consistente
    updateTextBox(boxId, {
      x: e.target.x(),
      y: e.target.y()
    })
  }}
}}
>

```

7. Mecânica de Inputs Controlados

Aqui está o **detalhe técnico** que faz a mágica acontecer:

Input de texto no painel:

```

<input
  // Valor SEMPRE vem do estado global
  value={run.text}

  // onChange atualiza estado global
  onChange={ (e) => {
    updateTextBoxRun(boxId, runIndex, { text: e.target.value })
  }}

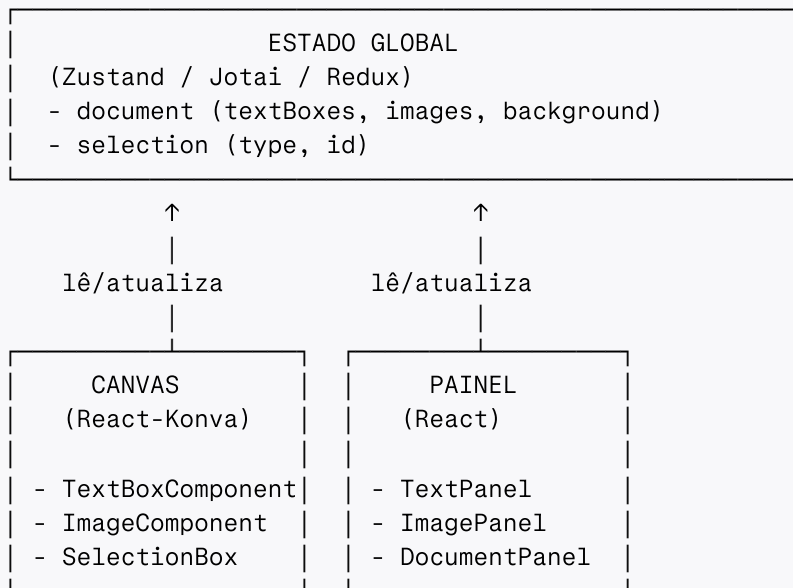
  // onBlur opcional: valida ou formata
  onBlur={ (e) => {
    // Remove espaços extras, valida, etc.
    const trimmed = e.target.value.trim()
    if (trimmed !== run.text) {
      updateTextBoxRun(boxId, runIndex, { text: trimmed })
    }
  }}

  // onKeyDown para shortcuts (Enter salva, Esc cancela)
  onKeyDown={ (e) => {
    if (e.key === "Escape") {
      // Restaura valor original do estado
      e.target.value = run.text
      e.target.blur()
    }
  }}
/>

```

Regra de ouro: O `value` do input **sempre** reflete o estado global. Nunca use estado local para inputs do painel.

8. Fluxo Completo (Diagrama Técnico)



FLUXO DE EDIÇÃO:

1. Usuário clica no canvas (`TextBoxComponent`)
→ `selectElement("box-1")`
→ `ESTADO.selection = { id: "box-1" }`
2. React notifica Painel
→ Painel lê `ESTADO.document.textBoxes.find("box-1")`
→ Painel preenche inputs
3. Usuário digita no input do Painel
→ `onChange` dispara `updateTextBoxRun("box-1", 0, { text: "P" })`
→ `ESTADO.document.textBoxes[0].runs[0].text = "P"`
4. React notifica Canvas
→ `TextBoxComponent` re-renderiza
→ `Konva.Text` atualiza com novo texto
5. Usuário vê mudança INSTANTANEAMENTE

9. Checklist Técnico de Implementação

- [] Estado global configurado (Zustand/Jotai/Redux);
- [] Ações definidas (`selectElement`, `updateTextBoxRun`, `addTextBox`, etc.);
- [] Canvas "assina" o estado (`useDocumentStore` em cada componente);
- [] Painel "assina" seleção + documento;
- [] Inputs do painel são controlados (`value` = estado, `onChange` = atualiza estado);
- [] Seleção limpa ao clicar no fundo (`onClick` no Stage);
- [] Bounding box reage à seleção (`SelectionBox` condicional);

- [] Drag-and-drop nativo do Konva (draggable + onDragEnd);
- [] Memoização em componentes pesados (React.memo);
- [] Seletores granulares (não assinar estado inteiro);
- [] Teste de fluidez (digitar rápido, arrastar, ver se trava).

10. Armadilhas Comuns e Como Evitar

Problema	Causa	Solução
Input "pula" caracteres	Estado local + global misturados	Use apenas estado global
Canvas não atualiza	Componente não "assina" estado	Use <code>useDocumentStore()</code> no componente
Digitação lenta	Re-renderização de tudo	Use seletores granulares
Bounding box some	Seleção não atualiza	Verifique se <code>selection.id</code> muda
Drag trava	Atualizar estado a cada pixel	Atualize apenas no <code>onDragEnd</code>
Painel não carrega elemento	<code>useEffect</code> sem dependências	Adicione <code>selection.id</code> nas deps

Resumo: A Fórmula da Fluidez

FLUIDEZ = Estado Único + Reatividade + Inputs Controlados

1. ESTADO ÚNICO:

- Tudo vive em um lugar (document + selection)
- Canvas e painel são apenas visualizações

2. REATIVIDADE:

- Quando estado muda, todos os "ouvintes" atualizam
- Use Zustand/Jotai/Redux para isso

3. INPUTS CONTROLADOS:

- value = estado global
- onChange = atualiza estado global
- Sem estado local em inputs

Se você seguir essa arquitetura, vai ter a **mesma fluidez do Canva**: editar no painel e ver acontecer no canvas instantaneamente, sem bugs de sincronização.

Quer que eu detalhe algum ponto específico (ex: estrutura exata do Zustand, otimizações de performance, ou exemplo de seletor granular)?

✱

1. <https://www.figma.com/blog/figmas-journey-to-typescript-compiling-away-our-custom-programming-language/>